1. Why Do We Need Unity Catalog?

As a Databricks environment grows, we may have:

- Multiple Data Engineering teams
- Multiple departments
- DEV and PROD environments
- Multiple Databricks workspaces
- Different access requirements
- Sensitive production datasets

Then questions appear:

- Who can read a production table?
- Who can modify it?
- Can permissions be managed centrally?
- Can multiple workspaces access governed data?
- Can we track where data came from?

Managing these things independently becomes difficult.

This is where **Unity Catalog** becomes important. Class 10 — Unity Catalog in Azu…

---

2. What Is Unity Catalog?

Unity Catalog is Databricks' centralized system for:

- Data governance
- Metadata management
- Access control
- Auditing
- Lineage
- Organizing data assets

A simple definition:

> **Unity Catalog provides centralized governance for data assets in Databricks.**

Example:

Data Engineer

→ May create/update tables

Analyst
→ May only read tables

Other User
→ May have no access

Unity Catalog helps control these permissions.

---

3. Unity Catalog vs Delta Lake

This is one of the most important distinctions.

Delta Lake

Handles:

- ACID transactions
- INSERT
- UPDATE
- DELETE
- MERGE
- Transaction history
- Time Travel
- RESTORE
- Delta data management

Unity Catalog

Handles:

- Catalog organization
- Schema organization
- Access control
- Permissions
- Metadata
- Governance
- Lineage
- Centralized management

Easy Way to Remember

Delta Lake
→ How the table works

Unity Catalog
→ Who can access and manage the table

Unity Catalog does **not** replace Delta Lake.

They work together. Class 10 — Unity Catalog in Azu…

---

4. Hive Metastore vs Unity Catalog

We previously worked with Hive Metastore.

Hive Metastore

Traditional structure:

database.table

Example:

datax_db.orders

Characteristics from this class:

- Traditional metadata system
- Legacy workspace-level metastore
- Manages database and table metadata
- More limited centralized governance
- Common in older Databricks environments

Unity Catalog

Uses:

catalog.schema.table

Example:

datax_catalog.bronze.orders

Provides:

- Centralized governance
- Fine-grained access control
- Lineage
- Auditing
- Governed tables and other assets
- Cross-workspace governance for supported attached workspaces

Class 10 — Unity Catalog in Azu…

---

5. Hive Metastore vs Unity Catalog — Quick Comparison

Hive Metastore	Unity Catalog
`database.table`	`catalog.schema.table`
Legacy approach	Modern centralized governance
Workspace-oriented	Can govern across attached workspaces
Metadata management	Metadata + governance
Limited centralized access control	Fine-grained access control
Older Databricks environments	Preferred for newer governed environments

---

6. Unity Catalog Three-Level Namespace

Unity Catalog uses:

Catalog
↓

Schema
↓
Table

Example:

datax_catalog.bronze.orders

Breakdown:

datax_catalog
→ Catalog

bronze
→ Schema

orders
→ Table

---

7. What Is a Catalog?

A catalog is the top-level logical container.

It can be organized by:

- Environment
- Department
- Business domain
- Project

Example:

datax_catalog

In companies, you may see designs such as:

dev_catalog
prod_catalog
finance_catalog
sales_catalog

---

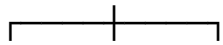
8. What Is a Schema?

A schema exists inside a catalog.

It helps organize related assets.

For example:

datax_catalog



bronze silver gold

Schemas can contain:

- Tables
- Views
- Other governed assets

---

9. What Is a Table?

A table contains structured data that can be queried using:

- SQL
- PySpark

Example:

datax_catalog.bronze.orders

Important

Catalogs and schemas are **logical governance objects**.

They should not automatically be thought of as physical storage folders.

Class 10 — Unity Catalog in Azu…

---

10. Exploring Unity Catalog

In Catalog Explorer, we can navigate:

Catalog
  ↓
Schema
  ↓
Table

Depending on permissions, we may also see other governed objects.

We can inspect metadata such as:

- Columns
- Storage location
- Ownership
- Permissions

---

11. Useful Unity Catalog SQL Commands

List visible catalogs:

SHOW CATALOGS;

Check current catalog:

SELECT current_catalog();

This matters because the current catalog affects how shorter names are resolved.

If there is any possibility of confusion, fully qualified names are safer.

---

12. Create a Catalog

CREATE CATALOG IF NOT EXISTS datax_catalog;

Then:

SHOW CATALOGS;

Select it:

USE CATALOG datax_catalog;

Important

Creating a catalog requires appropriate privileges.

In real companies, catalogs may be created by:

- Platform engineers
- Databricks administrators
- Data platform teams

A Data Engineer may simply receive access to an existing catalog.

---

13. Bronze, Silver and Gold Schemas

Create:

CREATE SCHEMA IF NOT EXISTS datax_catalog.bronze;

CREATE SCHEMA IF NOT EXISTS datax_catalog.silver;

CREATE SCHEMA IF NOT EXISTS datax_catalog.gold;

Verify:

SHOW SCHEMAS IN datax_catalog;

---

14. Medallion Organization

For this class:

Bronze
→ Raw / initially ingested data

Silver

→ Cleaned and validated data

Gold
→ Business-ready datasets

Creating these schemas does not automatically create a full Medallion pipeline.

You still need:

- Ingestion
- Validation
- Transformation
- Orchestration

---

15. Creating a Unity Catalog Table

The existing source table is:

hive_metastore.datax_db.orders

Create a UC table:

```
CREATE TABLE datax_catalog.bronze.orders_uc_demo
USING DELTA
AS
SELECT *
FROM hive_metastore.datax_db.orders;
```

Here:

Source
→ hive_metastore.datax_db.orders

Target
→ datax_catalog.bronze.orders_uc_demo

This creates a new table using CTAS.

---

16. Important: This Is Not a Full Migration

This operation copies data into a new Unity Catalog table.

It does not automatically migrate:

- Permissions
- Ownership
- All table settings
- Governance configuration

So:

CTAS Copy
≠
Complete Hive Metastore Migration

Class 10 — Unity Catalog in Azu…

---

17. Inspect the Unity Catalog Table

Query:

```
SELECT *
FROM datax_catalog.bronze.orders_uc_demo
ORDER BY order_id;
```

Inspect details:

```
DESCRIBE DETAIL datax_catalog.bronze.orders_uc_demo;
```

You can inspect:

- Format
- Location
- Table metadata

The table uses Delta for storage/transactions while Unity Catalog governs it.

---

18. Unity Catalog + Delta Lake Together

Consider:

datax_catalog.bronze.orders_uc_demo

Delta Lake handles:

Storage
Transactions
UPDATE
DELETE
MERGE
History

Unity Catalog handles:

Identity
Permissions
Governance
Metadata
Lineage

Together:

Governed Delta Table

---

19. DEV and PROD Organization

A common design is to separate environments using catalogs.

Example:

DEV

dev_catalog.bronze.orders
dev_catalog.silver.orders
dev_catalog.gold.daily_sales

PROD

prod_catalog.bronze.orders
prod_catalog.silver.orders
prod_catalog.gold.daily_sales

This makes it easier to control permissions separately.

---

20. Example Access Design

Developer:

DEV
→ Create / Modify

PROD
→ Limited access

Production service principal:

PROD
→ Required write permissions

Analyst:

Gold Tables
→ Read-only access

This reduces accidental production changes.

---

21. Environment Separation Is a Design Decision

Separate catalogs are one possible approach.

Organizations may also separate:

- Workspaces
- Storage accounts
- Deployment environments
- Other infrastructure

The correct design depends on:

- Security requirements
- Operational requirements
- Governance requirements

Class 10 — Unity Catalog in Azu…

---

22. Unity Catalog Permissions

Being able to see a table does not automatically mean you can query or modify it.

Unity Catalog uses privileges.

Inspect grants:

SHOW GRANTS ON TABLE
datax_catalog.bronze.orders_uc_demo;

---

23. Least Privilege Principle

A good security principle is:

> Give users only the permissions they actually need.

Example:

Analyst
→ SELECT

Pipeline Service Principal
→ Write/Modify permissions

Regular User
→ No unnecessary production access

Avoid giving everyone full access just because it is easier.

---

24. Troubleshooting Permission Errors

Suppose:

> A user can see the table in Catalog Explorer but cannot query it.

Check:

1. User identity
2. Group membership
3. `USE CATALOG`
4. `USE SCHEMA`
5. `SELECT` on table
6. Workspace bindings or other restrictions

This is a practical interview scenario. Class 10 — Unity Catalog in Azu…

---

25. Fully Qualified Names

Unity Catalog naming:

catalog.schema.table

Example:

```
SELECT *
FROM datax_catalog.bronze.orders_uc_demo;
```

Using fully qualified names helps avoid confusion between environments or similarly named tables.

---

26. Common Mistakes

Mistake 1 — Using Wrong Catalog

If you rely only on the current/default catalog, you may accidentally query the wrong environment.

Prefer fully qualified names when needed.

Mistake 2 — Giving Too Many Permissions

Avoid giving developers unnecessary access to PROD.

Follow:

Least Privilege

Mistake 3 — Assuming Unity Catalog Automatically Migrates Hive Tables

It does not.

Existing Hive Metastore tables require planned migration.

Mistake 4 — Treating Schemas Like Physical Folders

Catalogs and schemas are logical governance objects.

They are not simply storage directories.

Mistake 5 — Assuming Unity Catalog Replaces Delta Lake

It does not.

Delta Lake
→ Transactions

Unity Catalog
→ Governance

Class 10 — Unity Catalog in Azu…

---

27. Quick Revision

Hive Metastore
→ database.table

Unity Catalog
→ catalog.schema.table

Example:

Hive:
datax_db.orders

Unity Catalog:
datax_catalog.bronze.orders

---

28. Unity Catalog Hierarchy

Unity Catalog Metastore
↓
Catalog
↓
Schema
↓
Table

Example:

datax_catalog
↓
bronze
↓
orders

---

29. Unity Catalog vs Delta Lake — Quick Revision

Delta Lake	Unity Catalog
ACID transactions	Governance
INSERT/UPDATE/DELETE	Permissions
MERGE	Metadata management
History	Lineage
Time Travel	Auditing
RESTORE	Centralized access management

---

30. Student Practice

Task 1 — Explore Unity Catalog

Open Catalog Explorer.

Identify:

- Catalog
- Schema
- Table

Inspect the metadata.

Task 2 — Create Practice Catalog

If you have permission:

CREATE CATALOG IF NOT EXISTS datax_practice;

Create:

CREATE SCHEMA IF NOT EXISTS datax_practice.bronze;

CREATE SCHEMA IF NOT EXISTS datax_practice.silver;

CREATE SCHEMA IF NOT EXISTS datax_practice.gold;

If catalog creation is restricted, use an existing catalog where you have the required access.

Task 3 — Create a Delta Table

Create a small orders table inside:

datax_practice.bronze

Task 4 — Inspect Metadata

DESCRIBE DETAIL
datax_catalog.bronze.orders_uc_demo;

Inspect:

- Format
- Location
- Other table details

Task 5 — Explain These Differences

Practice explaining:

Hive Metastore vs Unity Catalog

Catalog vs Schema

Unity Catalog vs Delta Lake

DEV vs PROD Catalog Design

Class 10 — Unity Catalog in Azu…

---

31. Interview-Relevant Questions

1. What is Unity Catalog?

Unity Catalog is Databricks' centralized governance system for managing data access, metadata, lineage, and governed data assets.

---

2. What is the difference between Hive Metastore and Unity Catalog?

Hive Metastore is a legacy metadata system generally scoped to a workspace.

Unity Catalog provides centralized governance across supported attached workspaces and uses a three-level namespace.

---

3. What is the Unity Catalog three-level namespace?

catalog.schema.table

---

4. What is a Catalog?

A catalog is the top-level logical container used to organize governed data assets.

---

5. What is a Schema?

A schema is a logical container inside a catalog that organizes tables, views, and other assets.

---

6. Does Unity Catalog replace Delta Lake?

No.

Delta Lake provides transactional table functionality.

Unity Catalog provides governance and access control.

---

7. Can multiple Databricks workspaces access the same UC tables?

Yes, when the workspaces are attached to the appropriate Unity Catalog metastore and the required access is permitted.

Additional workspace bindings or restrictions may also apply.

---

8. Does enabling Unity Catalog automatically migrate Hive Metastore tables?

No.

Migration requires planning.

---

9. How would you troubleshoot a Unity Catalog permission issue?

Check:

- Identity
- Group membership
- `USE CATALOG`
- `USE SCHEMA`
- Table permissions
- Applicable workspace restrictions

Class 10 — Unity Catalog in Azu…

---

10. Why use fully qualified table names?

To make it clear exactly which catalog, schema, and table are being referenced.

Example:

prod_catalog.gold.daily_sales

---

11. How would you organize DEV and PROD using Unity Catalog?

One possible approach is:

dev_catalog
prod_catalog

with separate permissions for:

- Developers
- Production jobs
- Analysts

The exact design depends on security and operational requirements.

---

12. What does Unity Catalog provide beyond metadata?

It also provides capabilities such as:

- Access control
- Lineage
- Auditing
- Centralized governance

Key Takeaway

Growing Databricks Platform
↓
Multiple Teams
Multiple Workspaces
Multiple Environments
↓
Need Centralized Governance
↓
Unity Catalog
↓
Catalog
↓
Schema
↓
Table
↓
Permissions + Metadata + Lineage + Governance

The most important distinction to remember is:

Delta Lake makes tables transactional and reliable. Unity Catalog makes those data assets governed, organized, and securely accessible.